

Plantillas Técnicas

APIs, Python, SQL, Automatización, IA, Docker y Casos Prácticos

Javier Axayácatl Melchor Hernández

Índice

1. Objetivo del documento	5
2. Estructura recomendada para GitHub	5
3. README sugerido para GitHub	6
4. Python básico	6
4.1. Estructura mínima de un script	6
4.2. Variables	6
4.3. Condicional if	7
4.4. Ciclo for	7
4.5. Funciones	7
4.6. Listas	7
4.7. Diccionarios	7
4.8. Lista de diccionarios	8
5. Consumo de APIs con Python	8
5.1. Instalar requests	8
5.2. GET básico	8
5.3. Explicación del GET	8
5.4. POST básico	9
5.5. GET con headers	9
5.6. POST con API Key	9
5.7. Plantilla general para consumir una API	10
6. JSON	10
6.1. Ejemplo de JSON	10
6.2. JSON como diccionario en Python	11
6.3. Convertir diccionario a JSON	11
6.4. Convertir JSON a diccionario	11

7. SQL Server básico	11
7.1. Crear base de datos	11
7.2. Usar base de datos	11
7.3. Crear tabla	12
7.4. Insertar datos	12
7.5. Consultar datos	12
7.6. Consultar columnas específicas	12
7.7. Consultar con WHERE	12
7.8. Actualizar datos	12
7.9. Eliminar datos	12
7.10. CRUD completo	13
8. Relaciones y JOIN en SQL	13
8.1. Crear tabla relacionada	13
8.2. Insertar tickets	13
8.3. INNER JOIN	14
9. Conectar Python con SQL Server	14
9.1. Instalar pyodbc	14
9.2. Instalar el ODBC Driver	14
9.3. Conexión básica	14
9.4. Conexión usando Driver 18	15
9.5. Insertar desde Python	15
9.6. Manejo básico de errores	16
10. Mini API con Flask	16
10.1. Instalar Flask	16
10.2. API mínima	16
10.3. API que devuelve JSON	17
10.4. API con POST	17
11. Plantilla de API de IA	18
11.1. Estructura general	18
11.2. Nota importante	18
12. Docker básico	19
12.1. Definición corta	19
12.2. Conceptos clave	19
12.3. Dockerfile básico para Python	19
12.4. requirements.txt	19
12.5. Comandos básicos	19
12.6. Respuesta de entrevista	20
13. MongoDB y NoSQL básico	20
13.1. Diferencia entre SQL y NoSQL	20
13.2. Documento tipo MongoDB	20
13.3. Respuesta de entrevista	20

14. Automatización con IA	20
14.1. N8N	20
14.2. Make	21
14.3. Power Automate	21
14.4. Flujo típico de automatización	21
14.5. Caso: correos con IA	21
15. Prompt Engineering	22
15.1. Prompt malo	22
15.2. Prompt bueno	22
15.3. Estructura recomendada	22
16. Fine Tuning	23
16.1. Definición	23
16.2. Cuándo usarlo	23
16.3. Cuándo no usarlo	23
16.4. Respuesta de entrevista	23
17. ISO 27001	23
17.1. Definición	23
17.2. Pilares principales	23
17.3. IA e ISO 27001	24
17.4. Respuesta de entrevista	24
18. Casos prácticos posibles	24
18.1. Caso 1: Consumir una API pública	24
18.2. Caso 2: Enviar datos por POST	24
18.3. Caso 3: Crear una base de datos y una tabla	25
18.4. Caso 4: Insertar incidencias	25
18.5. Caso 5: Consultar incidencias pendientes	25
18.6. Caso 6: Actualizar estado	26
18.7. Caso 7: Conectar Python con SQL y leer datos	26
18.8. Caso 8: Clasificar texto con lógica simple	26
18.9. Caso 9: Simular clasificación de incidencia con IA	26
18.10 Caso 10: Convertir datos a JSON	27
19. Casos de IA para Transportes Lock	27
19.1. Caso A: Automatización documental	27
19.2. Flujo	27
19.3. Caso B: Sistema de tickets con IA	28
19.4. Caso C: Rutas y operación	28
19.5. Caso D: Tres meses y presupuesto limitado	28
19.6. Valor económico ejemplo	28

20.Preguntas técnicas rápidas	28
20.1. ¿Qué es una API?	28
20.2. ¿Qué es un endpoint?	29
20.3. ¿Qué es un request?	29
20.4. ¿Qué es un response?	29
20.5. ¿Qué es JSON?	29
20.6. ¿Qué es una API Key?	29
20.7. ¿Qué diferencia hay entre GET y POST?	29
20.8. ¿Qué es SQL?	29
20.9. ¿Qué es NoSQL?	29
20.10 ¿Qué es Docker?	29
20.11 ¿Qué es un LLM?	29
20.12 ¿Qué es Prompt Engineering?	30
20.13 ¿Qué es Fine Tuning?	30
20.14 ¿Qué es ISO 27001?	30
21.Cómo hablar de experiencia propia	30
21.1. Uso profesional de IA	30
21.2. Proyecto Gravyx	30
21.3. Borigen Studios	30
22.Checklist final	30
23.Comentarios Finales	31

1. Objetivo del documento

Este documento contiene plantillas rápidas para consultar durante una práctica técnica o entrevista. El objetivo no es memorizar todo, sino tener ejemplos organizados para resolver tareas comunes relacionadas con:

- Python básico.
- Consumo de APIs.
- Manejo de JSON.
- SQL Server básico.
- Conexión de Python con SQL Server.
- Automatización con IA.
- Docker básico.
- MongoDB / NoSQL básico.
- Casos prácticos de IA empresarial.

2. Estructura recomendada para GitHub

Si se sube este material a GitHub, se recomienda usar una estructura simple:

```
ai-interview-toolkit/  
|  
|-- README.md  
|-- docs/  
| |-- guia_plantillas.pdf  
| |-- guia_plantillas.tex  
|  
|-- python/  
| |-- 01_basico.py  
| |-- 02_api_get.py  
| |-- 03_api_post.py  
| |-- 04_json.py  
|  
|-- sql/  
| |-- 01_crear_base.sql  
| |-- 02_crud.sql  
| |-- 03_joins.sql  
|  
|-- python_sql/  
| |-- conexion_sql.py  
|  
|-- docker/
```

```
| |-- Dockerfile
| |-- comandos_basicos.md
|
|-- ia/
| |-- prompts.md
| |-- casos_uso.md
```

3. README sugerido para GitHub

```
# AI Interview Toolkit

Repositorio personal de prctica para entrevista tcnica de Especialista en IA.

Incluye ejemplos bsicos de:

- Python
- Consumo de APIs
- JSON
- SQL Server
- Conexin Python con SQL Server
- Docker bsico
- Automatizacin con IA
- Casos prcticos empresariales

Este repositorio funciona como material de estudio y referencia rpida.
```

4. Python bsico

4.1. Estructura mnima de un script

```
# Archivo: 01_basico.py

def main():
    print("Hola, este es mi primer script en Python")

if __name__ == "__main__":
    main()
```

Idea clave: la funcin `main()` permite organizar el programa. La lnea `if __name__ == "__main__":` indica que el programa inicia desde ah.

4.2. Variables

```
nombre = "Javier"
edad = 29
```

```
activo = True

print(nombre)
print(edad)
print(activo)
```

4.3. Condicional if

```
edad = 29

if edad >= 18:
    print("Es mayor de edad")
else:
    print("Es menor de edad")
```

4.4. Ciclo for

```
for i in range(5):
    print("Numero:", i)
```

4.5. Funciones

```
def sumar(a, b):
    resultado = a + b
    return resultado

print(sumar(5, 3))
```

4.6. Listas

```
usuarios = ["Adriel", "Juan", "Ana"]

for usuario in usuarios:
    print(usuario)
```

4.7. Diccionarios

```
usuario = {
    "nombre": "Adriel",
    "correo": "adriel@gmail.com"
}

print(usuario["nombre"])
```

```
print(usuario["correo"])
```

4.8. Lista de diccionarios

```
usuarios = [  
    {"nombre": "Adriel", "correo": "adriel@gmail.com"},  
    {"nombre": "Juan", "correo": "juan@gmail.com"}  
]  
  
for usuario in usuarios:  
    print(usuario["nombre"], usuario["correo"])
```

5. Consumo de APIs con Python

5.1. Instalar requests

```
pip install requests
```

5.2. GET básico

```
# Archivo: 02_api_get.py  
  
import requests  
  
url = "https://jsonplaceholder.typicode.com/users"  
  
respuesta = requests.get(url)  
  
print("cdigo de estado: ", respuesta.status_code)  
print()  
  
usuarios = respuesta.json()  
  
for usuario in usuarios:  
    print("Nombre", usuario["name"])  
  
print("-----")
```

5.3. Explicación del GET

- `requests.get(url)` envía una petición GET.
- `status_code` indica si la petición fue exitosa.

- `respuesta.json()` convierte la respuesta JSON a objetos de Python.
- El ciclo `for` recorre cada usuario.

5.4. POST básico

```
# Archivo: 03_api_post.py

import requests

url = "https://jsonplaceholder.typicode.com/posts"

datos = {
    "title": "Reporte de prueba",
    "body": "Este es un contenido enviado desde Python",
    "userId": 1
}

respuesta = requests.post(url, json=datos)

print("Codigo de estado:", respuesta.status_code)
print(respuesta.json())
```

5.5. GET con headers

```
import requests

url = "https://api.ejemplo.com/datos"

headers = {
    "Authorization": "Bearer TU_API_KEY",
    "Content-Type": "application/json"
}

respuesta = requests.get(url, headers=headers)

print(respuesta.status_code)
print(respuesta.json())
```

5.6. POST con API Key

```
import requests

url = "https://api.ejemplo.com/procesar"

headers = {
```

```

    "Authorization": "Bearer TU_API_KEY",
    "Content-Type": "application/json"
}

datos = {
    "mensaje": "Analiza este texto"
}

respuesta = requests.post(url, headers=headers, json=datos)

print(respuesta.status_code)
print(respuesta.json())

```

5.7. Plantilla general para consumir una API

```

import requests

url = "URL_DEL_ENDPOINT"

headers = {
    "Authorization": "Bearer API_KEY",
    "Content-Type": "application/json"
}

payload = {
    "campo": "valor"
}

response = requests.post(url, headers=headers, json=payload)

if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print("Error:", response.status_code)
    print(response.text)

```

6. JSON

6.1. Ejemplo de JSON

```

{
    "nombre": "Adriel",
    "correo": "adriel@gmail.com",
    "activo": true
}

```

6.2. JSON como diccionario en Python

```
usuario = {  
    "nombre": "Adriel",  
    "correo": "adriel@gmail.com",  
    "activo": True  
}  
  
print(usuario["nombre"])
```

6.3. Convertir diccionario a JSON

```
import json  
  
usuario = {  
    "nombre": "Adriel",  
    "correo": "adriel@gmail.com"  
}  
  
usuario_json = json.dumps(usuario, indent=4)  
  
print(usuario_json)
```

6.4. Convertir JSON a diccionario

```
import json  
  
texto_json = '{"nombre": "Adriel", "correo": "adriel@gmail.com"}'  
  
usuario = json.loads(texto_json)  
  
print(usuario["nombre"])
```

7. SQL Server básico

7.1. Crear base de datos

```
CREATE DATABASE mitablaAPI;
```

7.2. Usar base de datos

```
USE mitablaAPI;
```

7.3. Crear tabla

```
CREATE TABLE Usuario(  
    id INT IDENTITY(1,1) PRIMARY KEY,  
    nombre VARCHAR(100),  
    correo VARCHAR(100)  
);
```

7.4. Insertar datos

```
INSERT INTO Usuario(nombre, correo)  
VALUES  
( 'Adriel', 'adriel@gmail.com'),  
( 'Juan', 'juan@gmail.com');
```

7.5. Consultar datos

```
SELECT * FROM Usuario;
```

7.6. Consultar columnas específicas

```
SELECT id, nombre, correo  
FROM Usuario;
```

7.7. Consultar con WHERE

```
SELECT * FROM Usuario  
WHERE nombre = 'Adriel';
```

7.8. Actualizar datos

```
UPDATE Usuario  
SET correo = 'nuevo_correo@gmail.com'  
WHERE id = 1;
```

7.9. Eliminar datos

```
DELETE FROM Usuario  
WHERE id = 2;
```

7.10. CRUD completo

CRUD significa:

- Create: crear o insertar datos.
- Read: leer o consultar datos.
- Update: actualizar datos.
- Delete: eliminar datos.

```
-- CREATE
INSERT INTO Usuario(nombre, correo)
VALUES ('Pedro', 'pedro@gmail.com');

-- READ
SELECT * FROM Usuario;

-- UPDATE
UPDATE Usuario
SET nombre = 'Pedro Lpez'
WHERE id = 3;

-- DELETE
DELETE FROM Usuario
WHERE id = 3;
```

8. Relaciones y JOIN en SQL

8.1. Crear tabla relacionada

```
CREATE TABLE Ticket(
  id INT IDENTITY(1,1) PRIMARY KEY,
  usuario_id INT,
  descripcion VARCHAR(255),
  estado VARCHAR(50),
  FOREIGN KEY (usuario_id) REFERENCES Usuario(id)
);
```

8.2. Insertar tickets

```
INSERT INTO Ticket(usuario_id, descripcion, estado)
VALUES
(1, 'No funciona el sistema', 'pendiente'),
(1, 'Error en reporte', 'resuelto');
```

8.3. INNER JOIN

```
SELECT
    Usuario.nombre,
    Ticket.descripcion,
    Ticket.estado
FROM Ticket
INNER JOIN Usuario
ON Ticket.usuario_id = Usuario.id;
```

Idea clave: INNER JOIN une dos tablas cuando existe coincidencia entre ambas.

9. Conectar Python con SQL Server

9.1. Instalar pyodbc

```
pip install pyodbc
```

9.2. Instalar el ODBC Driver

Para que Python pueda conectarse con SQL Server se necesita:

Microsoft ODBC Driver for SQL Server

Se puede instalar el Driver 17 o Driver 18. Después de instalarlo, se puede verificar desde Python:

```
import pyodbc

print(pyodbc.drivers())
```

Si aparece una salida como:

```
ODBC Driver 17 for SQL Server
```

o

```
ODBC Driver 18 for SQL Server
```

entonces el driver está instalado.

9.3. Conexión básica

```
# Archivo: conexion_sql.py

import pyodbc

servidor = "localhost\\SQL2025"
base_datos = "mitablaAPI"
```

```

conexion = pyodbc.connect(
    "DRIVER={ODBC DRIVER 17 for SQL SERVER};"
    f"SERVER={servidor};"
    f"DATABASE={base_datos};"
    "Trusted_connection=yes;"
)

cursor = conexion.cursor()

cursor.execute("SELECT id, nombre, correo FROM Usuario")
usuarios = cursor.fetchall()

for usuario in usuarios:
    print("id", usuario.id)
    print("nombre", usuario.nombre)
    print("correo", usuario.correo)
    print("-----")

cursor.close()
conexion.close()

```

9.4. Conexión usando Driver 18

En algunos equipos el driver instalado puede ser el 18:

```

conexion = pyodbc.connect(
    "DRIVER={ODBC Driver 18 for SQL Server};"
    f"SERVER={servidor};"
    f"DATABASE={base_datos};"
    "Trusted_Connection=yes;"
    "TrustServerCertificate=yes;"
)

```

9.5. Insertar desde Python

```

import pyodbc

servidor = "localhost\\SQL2025"
base_datos = "mitablaAPI"

conexion = pyodbc.connect(
    "DRIVER={ODBC DRIVER 17 for SQL SERVER};"
    f"SERVER={servidor};"
    f"DATABASE={base_datos};"
    "Trusted_connection=yes;"
)

```

```

cursor = conexion.cursor()

nombre = "Ana"
correo = "ana@gmail.com"

cursor.execute(
    "INSERT INTO Usuario(nombre, correo) VALUES (?, ?)",
    nombre,
    correo
)

conexion.commit()

cursor.close()
conexion.close()

```

9.6. Manejo básico de errores

```

import pyodbc

try:
    conexion = pyodbc.connect(
        "DRIVER={ODBC DRIVER 17 for SQL SERVER};"
        "SERVER=localhost\\SQL2025;"
        "DATABASE=mitablaAPI;"
        "Trusted_connection=yes;"
    )

    print("Conexin exitosa")

except Exception as error:
    print("Ocurri un error:")
    print(error)

```

10. Mini API con Flask

10.1. Instalar Flask

```

pip install flask

```

10.2. API mínima

```

# Archivo: app.py

```



```

from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Hola, API funcionando"

if __name__ == "__main__":
    app.run(debug=True)

```

10.3. API que devuelve JSON

```

from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/usuarios")
def obtener_usuarios():
    usuarios = [
        {"id": 1, "nombre": "Adriel"},
        {"id": 2, "nombre": "Juan"}
    ]

    return jsonify(usuarios)

if __name__ == "__main__":
    app.run(debug=True)

```

10.4. API con POST

```

from flask import Flask, request, jsonify

app = Flask(__name__)

usuarios = []

@app.route("/usuarios", methods=["POST"])
def crear_usuario():
    datos = request.get_json()

    usuarios.append(datos)

    return jsonify({
        "mensaje": "Usuario creado",
        "usuario": datos
    }), 201

```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

11. Plantilla de API de IA

11.1. Estructura general

```
import requests  
  
API_KEY = "TU_API_KEY"  
  
url = "https://api.openai.com/v1/chat/completions"  
  
headers = {  
    "Authorization": f"Bearer {API_KEY}",  
    "Content-Type": "application/json"  
}  
  
data = {  
    "model": "gpt-4o-mini",  
    "messages": [  
        {  
            "role": "system",  
            "content": "Eres un asistente que resume reportes."  
        },  
        {  
            "role": "user",  
            "content": "Resume este reporte en 5 puntos."  
        }  
    ]  
}  
  
response = requests.post(url, headers=headers, json=data)  
  
print(response.status_code)  
print(response.json())
```

11.2. Nota importante

En una entrevista no es necesario memorizar exactamente todos los parámetros de una API comercial. Lo importante es entender:

- URL o endpoint.
- Headers.

- API Key.
- Payload o cuerpo de la petición.
- Respuesta JSON.

12. Docker básico

12.1. Definición corta

Docker permite empaquetar una aplicación junto con sus dependencias para ejecutarla de forma consistente en diferentes entornos.

12.2. Conceptos clave

- **Imagen:** plantilla con la aplicación y dependencias.
- **Contenedor:** instancia en ejecución de una imagen.
- **Dockerfile:** archivo que define cómo construir la imagen.

12.3. Dockerfile básico para Python

```
FROM python:3.12

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

12.4. requirements.txt

```
flask
requests
pyodbc
```

12.5. Comandos básicos

```
docker build -t mi-api-python .
```

```
docker run -p 5000:5000 mi-api-python
```

12.6. Respuesta de entrevista

Docker es una herramienta de contenedorización que permite empaquetar una aplicación con sus dependencias y configuración para que funcione igual en desarrollo, pruebas o producción. No lo he implementado extensamente en producción, pero conozco el concepto y puedo aprenderlo rápidamente.

13. MongoDB y NoSQL básico

13.1. Diferencia entre SQL y NoSQL

SQL	NoSQL
Usa tablas	Usa documentos o estructuras flexibles
Tiene esquemas más rígidos	Tiene esquemas más flexibles
Ejemplo: SQL Server, MySQL, PostgreSQL	Ejemplo: MongoDB
Ideal para datos relacionales	Ideal para datos semiestructurados

13.2. Documento tipo MongoDB

```
{
  "_id": 1,
  "nombre": "Adriel",
  "correo": "adriel@gmail.com",
  "rol": "admin"
}
```

13.3. Respuesta de entrevista

No he usado MongoDB extensamente, pero entiendo que es una base de datos NoSQL orientada a documentos. A diferencia de SQL, no trabaja principalmente con tablas relacionadas, sino con documentos flexibles similares a JSON.

14. Automatización con IA

14.1. N8N

N8N es una plataforma de automatización basada en nodos. Permite conectar servicios, APIs, correos, bases de datos y modelos de IA.

14.2. Make

Make es una plataforma visual de automatización. Es útil para conectar aplicaciones como formularios, correos, hojas de cálculo y servicios externos.

14.3. Power Automate

Power Automate es la plataforma de automatización de Microsoft. Conviene cuando la empresa usa Outlook, Teams, Excel, SharePoint o Microsoft 365.

14.4. Flujo típico de automatización

```
Correo recibido  
  
Extraer contenido  
  
Enviar a modelo de IA  
  
Clasificar prioridad  
  
Guardar en base de datos  
  
Notificar por Teams, correo o WhatsApp
```

14.5. Caso: correos con IA

```
Entrada:  
- Correo de un usuario.  
- Documento adjunto.  
- Reporte o incidencia.  
  
Proceso:  
- Extraer texto.  
- Analizar con LLM.  
- Clasificar tema.  
- Asignar prioridad.  
- Enviar al rea correspondiente.  
  
Salida:  
- Resumen ejecutivo.  
- Registro en base de datos.  
- Notificación automática.
```

15. Prompt Engineering

15.1. Prompt malo

Analiza este reporte.

15.2. Prompt bueno

Acta como analista de operaciones.

Analiza el siguiente reporte de incidencia.

Extrae:

- Tipo de incidencia
- Nivel de prioridad
- rea responsable
- Riesgo operativo
- Resumen en mximo 5 lneas

Devuelve la respuesta en formato JSON.

15.3. Estructura recomendada

Rol:

Acta como...

Objetivo:

Tu tarea es...

Contexto:

La empresa necesita...

Instrucciones:

1. Analiza...
2. Clasifica...
3. Resume...

Formato de salida:

Devuelve una tabla / JSON / lista.

Restricciones:

No inventes informacin.

Si falta un dato, indica "No disponible".

16. Fine Tuning

16.1. Definición

Fine Tuning es el proceso de ajustar un modelo preentrenado usando datos específicos para especializarlo en una tarea o dominio.

16.2. Cuándo usarlo

- Cuando se necesita que el modelo responda con un estilo muy específico.
- Cuando existen muchos ejemplos históricos bien etiquetados.
- Cuando el prompt no es suficiente para mantener consistencia.

16.3. Cuándo no usarlo

- Cuando el problema puede resolverse con buen Prompt Engineering.
- Cuando hay pocos datos.
- Cuando la información cambia constantemente.
- Cuando conviene usar RAG o consulta a documentos.

16.4. Respuesta de entrevista

Primero intentaría resolver el caso con Prompt Engineering, automatización o recuperación de información. Consideraría Fine Tuning sólo si la empresa cuenta con suficientes datos históricos de calidad y si necesita un comportamiento muy especializado del modelo.

17. ISO 27001

17.1. Definición

ISO 27001 es un estándar internacional para la gestión de la seguridad de la información.

17.2. Pilares principales

- **Confidencialidad:** sólo acceden las personas autorizadas.
- **Integridad:** la información no debe modificarse indebidamente.
- **Disponibilidad:** la información debe estar accesible cuando se necesita.

17.3. IA e ISO 27001

Al implementar IA en una empresa se debe cuidar:

- No exponer información sensible.
- Controlar accesos.
- Registrar actividad.
- Evitar fuga de datos.
- Definir qué información puede enviarse a modelos externos.
- Coordinar con Seguridad TI.

17.4. Respuesta de entrevista

Al implementar IA en una empresa de seguridad o traslado de valores, lo primero sería cuidar la confidencialidad de la información. No enviaría datos sensibles a herramientas externas sin autorización. También definiría permisos, trazabilidad, registros de actividad y controles alineados con las políticas de Seguridad TI e ISO 27001.

18. Casos prácticos posibles

18.1. Caso 1: Consumir una API pública

Problema: Consume una API pública y muestra los nombres de usuarios.

```
import requests

url = "https://jsonplaceholder.typicode.com/users"

response = requests.get(url)

if response.status_code == 200:
    usuarios = response.json()

    for usuario in usuarios:
        print(usuario["name"])
else:
    print("Error:", response.status_code)
```

18.2. Caso 2: Enviar datos por POST

Problema: Envía datos a una API.


```

import requests

url = "https://jsonplaceholder.typicode.com/posts"

nuevo_post = {
    "title": "Nuevo reporte",
    "body": "Contenido del reporte",
    "userId": 1
}

response = requests.post(url, json=nuevo_post)

print(response.status_code)
print(response.json())

```

18.3. Caso 3: Crear una base de datos y una tabla

```

CREATE DATABASE EmpresaIA;

USE EmpresaIA;

CREATE TABLE Incidencia(
    id INT IDENTITY(1,1) PRIMARY KEY,
    titulo VARCHAR(100),
    descripcion VARCHAR(255),
    prioridad VARCHAR(50),
    estado VARCHAR(50)
);

```

18.4. Caso 4: Insertar incidencias

```

INSERT INTO Incidencia(titulo, descripcion, prioridad, estado)
VALUES
('Falla en sistema', 'No permite iniciar sesin', 'Alta', 'Pendiente'),
('Reporte duplicado', 'Se gener dos veces', 'Media', 'En revisin');

```

18.5. Caso 5: Consultar incidencias pendientes

```

SELECT *
FROM Incidencia
WHERE estado = 'Pendiente';

```

18.6. Caso 6: Actualizar estado

```
UPDATE Incidencia
SET estado = 'Resuelto'
WHERE id = 1;
```

18.7. Caso 7: Conectar Python con SQL y leer datos

```
import pyodbc

conexion = pyodbc.connect(
    "DRIVER={ODBC DRIVER 17 for SQL SERVER};"
    "SERVER=localhost\\SQL2025;"
    "DATABASE=EmpresaIA;"
    "Trusted_connection=yes;"
)

cursor = conexion.cursor()

cursor.execute("SELECT id, titulo, prioridad FROM Incidencia")

incidencias = cursor.fetchall()

for incidencia in incidencias:
    print(incidencia.id, incidencia.titulo, incidencia.prioridad)

cursor.close()
conexion.close()
```

18.8. Caso 8: Clasificar texto con lógica simple

```
texto = "No funciona el sistema de acceso"

if "no funciona" in texto.lower():
    prioridad = "Alta"
else:
    prioridad = "Media"

print("Prioridad:", prioridad)
```

18.9. Caso 9: Simular clasificación de incidencia con IA

```
def clasificar_incidencia(texto):
    texto = texto.lower()
```

```

if "urgente" in texto or "no funciona" in texto:
    return "Alta"
elif "revisin" in texto or "lento" in texto:
    return "Media"
else:
    return "Baja"

incidencia = "Urgente: no funciona el sistema de reportes"

prioridad = clasificar_incidencia(incidencia)

print("Prioridad detectada:", prioridad)

```

18.10. Caso 10: Convertir datos a JSON

```

import json

incidencia = {
    "titulo": "Falla en sistema",
    "prioridad": "Alta",
    "estado": "Pendiente"
}

incidencia_json = json.dumps(incidencia, indent=4)

print(incidencia_json)

```

19. Casos de IA para Transportes Lock

19.1. Caso A: Automatización documental

Respuesta modelo:

Implementaría un flujo de automatización documental con N8N o Power Automate. El sistema recibiría correos, reportes o documentos, extraería el contenido, lo enviaría a un modelo de lenguaje para generar un resumen, clasificaría la prioridad y asignaría el caso al área correspondiente. Esto reduciría tiempo administrativo, errores de clasificación y mejoraría la trazabilidad.

19.2. Flujo

```

Correo o documento

Extraccin de texto

```

Modelo de IA

Resumen y clasificacin

Registro en base de datos

Notificacin al responsable

19.3. Caso B: Sistema de tickets con IA

Implementaría un sistema centralizado de tickets e incidencias asistido por IA. Los usuarios podrían registrar incidencias mediante formulario, correo o imagen. El modelo clasificaría la incidencia, asignaría prioridad, sugeriría área responsable y generaría resúmenes ejecutivos para supervisores.

19.4. Caso C: Rutas y operación

Además de integrar servicios de geolocalización como Google Maps, exploraría modelos de optimización que consideren tráfico, clima, restricciones operativas e incidencias de seguridad. El objetivo no sería sólo encontrar la ruta más corta, sino la más eficiente y segura.

19.5. Caso D: Tres meses y presupuesto limitado

Empezaría con un proyecto de automatización documental e incidencias internas porque tiene bajo costo, impacto rápido y riesgo controlado. Mediría el éxito con indicadores como horas administrativas ahorradas, reducción del tiempo de respuesta, disminución de errores de clasificación y número de incidencias procesadas automáticamente.

19.6. Valor económico ejemplo

Si cinco personas dedican una hora diaria a clasificar correos y reportes, eso equivale aproximadamente a 100 horas mensuales. Si una automatización reduce ese tiempo en 70 %, se ahorrarían cerca de 70 horas al mes. Con un costo aproximado de 150 pesos por hora, el ahorro sería de 10,500 pesos mensuales, o más de 120,000 pesos al año.

20. Preguntas técnicas rápidas

20.1. ¿Qué es una API?

Una API es una interfaz que permite que dos aplicaciones se comuniquen e intercambien información de manera estructurada.

20.2. ¿Qué es un endpoint?

Es una dirección específica dentro de una API que permite acceder a un recurso.

20.3. ¿Qué es un request?

Es la petición que envía el cliente al servidor.

20.4. ¿Qué es un response?

Es la respuesta que devuelve el servidor.

20.5. ¿Qué es JSON?

JSON es un formato ligero de intercambio de datos basado en pares clave-valor.

20.6. ¿Qué es una API Key?

Es una credencial que permite autenticar y autorizar el acceso a una API.

20.7. ¿Qué diferencia hay entre GET y POST?

GET se usa para obtener información. POST se usa para enviar información y crear un recurso.

20.8. ¿Qué es SQL?

SQL es un lenguaje utilizado para crear, consultar, actualizar y eliminar información en bases de datos relacionales.

20.9. ¿Qué es NoSQL?

NoSQL es un tipo de base de datos flexible que no depende principalmente de tablas relacionales. MongoDB, por ejemplo, usa documentos similares a JSON.

20.10. ¿Qué es Docker?

Docker permite empaquetar una aplicación con sus dependencias para ejecutarla de forma consistente en distintos entornos.

20.11. ¿Qué es un LLM?

Un LLM es un modelo de lenguaje entrenado con grandes volúmenes de texto, capaz de generar respuestas, resumir información, analizar documentos y apoyar tareas de automatización.

20.12. ¿Qué es Prompt Engineering?

Es el diseño estructurado de instrucciones para obtener respuestas más precisas y útiles de un modelo de IA.

20.13. ¿Qué es Fine Tuning?

Es el proceso de ajustar un modelo preentrenado con datos específicos para especializarlo en una tarea concreta.

20.14. ¿Qué es ISO 27001?

Es un estándar internacional para la gestión de la seguridad de la información.

21. Cómo hablar de experiencia propia

21.1. Uso profesional de IA

Utilizo IA principalmente como acelerador de trabajo. Cuando trabajo sobre tecnologías que ya conozco, la uso para generar propuestas de implementación, documentación, código repetitivo o análisis de errores. Cuando la uso para tecnologías nuevas, valido los resultados con pruebas propias y documentación técnica.

21.2. Proyecto Gravyx

Uno de los proyectos más interesantes fue apoyar en el análisis de una plataforma llamada Gravyx. El principal reto era que la aplicación había sido desarrollada con herramientas de IA generativa y tenía poca documentación técnica. Para identificar errores fue necesario revisar componentes, entidades, flujos y dependencias prácticamente archivo por archivo. La IA ayudó a acelerar el análisis, pero cada hipótesis debía validarse manualmente antes de proponer cambios.

21.3. Borigen Studios

Borigen Studios trabaja por proyecto para distintas empresas. Mi participación se ha enfocado en desarrollo de software, integración de APIs, automatización y procesamiento de datos. He trabajado con backend, SQL, Python, análisis de datos y apoyo en debugging de aplicaciones.

22. Checklist final

Estado	Actividad
	Explicar qué es una API.
	Hacer un GET en Python.
	Hacer un POST en Python.

	Leer JSON en Python.
	Crear una base de datos en SQL Server.
	Crear una tabla.
	Hacer INSERT, SELECT, UPDATE y DELETE.
	Conectar Python con SQL Server usando pyodbc.
	Explicar Docker básico.
	Explicar SQL vs NoSQL.
	Explicar ISO 27001.
	Diseñar una automatización con IA.
	Explicar un proyecto propio usando IA.
	Proponer una solución de IA para Transportes Lock.

23. Comentarios Finales

Mi perfil combina formación científica, experiencia en desarrollo de software y uso práctico de inteligencia artificial. No me considero únicamente usuario de herramientas de IA, sino alguien que puede analizar procesos, identificar oportunidades de automatización, proponer soluciones técnicas y validar que generen valor real para la empresa.